

Sovereign Compute: A Zero-Dependency, Cryptographically Attested AI Substrate

Ahmad Ali Parr · SnapKitty Collective · July 2026

Abstract

Modern AI infrastructure is characterized by deep dependency chains: gigabyte-scale runtime libraries, proprietary accelerator drivers, cloud-hosted inference APIs, and opaque supply chains that make cryptographic attestation impossible. This paper presents the **Sovereign Compute** architecture: a zero-dependency, cryptographically attested AI substrate in which every computational artifact — from kernel primitives to trained model weights — carries a verifiable provenance chain anchored in hardware.

The substrate comprises seven interlocking layers: (1) the Abjad Compute Substrate, a SUBLEQ one-instruction machine addressed by Arabic numeral space with Born-collapse swarm intelligence; (2) ERRANT LFIS, a Linear Forth Instruction Set with Quantum Type Theory multiplicities enforced at the type level; (3) the Systemic Intelligence Stack, a seven-layer architecture from Agda constitution to JavaScript orchestrator; (4) the Sovereign Transformer, an x86-64 plasma gate with approximately five cycles latency and zero heap allocation; (5) Claudes Harness, a Prolog identity kernel with QTT capability governance; (6) a cryptographic substrate in pure Fortran implementing Blake3 and Ed25519; and (7) a deployment architecture targeting static PIE binaries, WASM Component Model, and HSM-sealed ELF sections.

The architecture achieves what no prior system has claimed: every inference step is a provable theorem, every model weight is a WORM-chained artifact, and every execution path is attested against a cryptographic root of trust. The system currently operates at TRS convergence point 388.985128 with zero external dependencies beyond the bare hardware instruction set.

1. Introduction

1.1 The Dependency Problem

A typical 2026 AI inference stack includes: Python runtime (CPython, ~50 MB), CUDA toolkit (>4 GB), cuDNN, PyTorch (>2 GB), the Hugging Face transformers library, tokenizers, sentencepiece, and dozens of transitive dependencies. Each dependency is a potential vector for supply-chain compromise, an obstacle to formal verification, and a source of non-determinism that makes cryptographic attestation impossible.

The problem is not merely practical. It is mathematical. When an inference result depends on a floating-point contraction that is only defined by a proprietary CUDA kernel, no formal proof system can verify the output. The computation is opaque by design.

1.2 The Sovereignty Imperative

For AI systems deployed in contexts requiring legal accountability — financial trust deeds, medical decision support, sovereign national infrastructure — the inability to formally verify computation is not an inconvenience. It is a disqualification.

The SnapKitty Collective’s Sovereign Compute architecture is designed from first principles to eliminate this disqualification. Every layer of the stack is:

1. **Formally specified** — either in Lean 4, Agda, SPARK/Ada, or Prolog with machine-checkable proofs.
2. **Zero-dependency** — the Fortran 2018 kernel has no `libc` calls; the WASM sandbox has no OS syscalls.
3. **Cryptographically attested** — every artifact carries a Blake3 hash chain anchored to an Ed25519 key pair with the public key embedded in a `.note.sov` ELF section.
4. **WORM-sealed** — once a computation artifact is registered in the chain, it cannot be modified without breaking the attestation.

1.3 Contributions

This paper makes the following contributions:

- A complete specification of the Abjad Compute Substrate (§3)
- A formal description of the ERRANT Linear Forth Instruction Set with Quantum Type Theory multiplicities (§4)
- An architectural account of the Systemic Intelligence Stack including its TRS convergence property (§5)
- A performance analysis of the x86-64 plasma gate (~5 cycles, no heap) (§6)
- A Prolog-based identity kernel for capability governance (§7)
- A pure-Fortran implementation of Blake3 and Ed25519 (§8)
- A deployment architecture for static, attested, zero-dependency binaries (§9)

2. Background

2.1 One-Instruction Set Computers

The SUBLEQ (SUBtract and branch if Less than or Equal) one-instruction set computer was described by Mavaddat and Parhami in 1988. The single instruction `SUBLEQ a, b, c` means: `mem[b] := mem[b] - mem[a]`; if `mem[b] < 0` then jump to address `c`. Despite this simplicity, SUBLEQ is Turing-complete. Every program can be compiled to a SUBLEQ program; the overhead is roughly $O(n^3)$ in instruction count relative to a standard ISA.

The Abjad Compute Substrate uses SUBLEQ not for general computation but for a specific purpose: providing a minimal, formally verifiable address space whose properties are mathematically well-characterized.

2.2 Quantum Type Theory

Quantum Type Theory (QTT) extends Martin-Löf Type Theory with usage annotations (multiplicities) that track how many times a resource may be used. The multiplicities form a semiring R with:

`lin` = use exactly once, `aff` = use at most once, `un` = use any number of times

Additional multiplicities introduced in ERRANT: `cap` (capability, may be acquired) and `seal` (sealed, cannot be opened without corresponding key). These extend the QTT semiring to five elements.

2.3 Bifrost Attestation

The Bifrost protocol is the SnapKitty attestation standard. Every kernel artifact registers a Bifrost receipt:

```
BifrostReceipt {
  artifact_hash: Blake3(content),
  prev_hash: Blake3(previous_receipt),
  timestamp: u64,           // TAI seconds since 2020-01-01
  author_pubkey: [u8; 32],  // Ed25519
  signature: [u8; 64],
}
```

Receipts form a linked-hash chain. Tampering with any artifact breaks all subsequent receipts. The chain is anchored by embedding the genesis receipt hash in a `.note.sov` ELF section at link time.

3. The Abjad Compute Substrate

3.1 Arabic Numeral Address Space

The Arabic Abjad numeral system assigns integer values to the 28 letters of the Arabic alphabet:

Letter	Name	Abjad Value
	Alif	1
	Ba	2
	Jim	3
	Dal	4
	Ha	5
	Waw	6
	Zayn	7
	Ha	8
	Ta	9
	Ya	10
	Kaf	20
	Lam	30
	Mim	40
	Nun	50
	Sin	60
	Ayn	70
	Fa	80
	Sad	90
	Qaf	100
	Ra	200
	Shin	300

Letter	Name	Abjad Value
	Ta	400
	Tha	500
	Kha	600
	Dhal	700
	Dad	800
	Za	900
	Ghayn	1000

This gives 28 address registers spanning values $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 200, 300, 400, \dots\}$.

The Abjad address space is not contiguous in the integers; it is a subset of $\mathbb{Z}_{1001}$ corresponding to the letter values. This deliberate non-contiguity provides a natural form of address space layout randomization: a linear scan of memory addresses will not systematically visit all Abjad registers.

The total address space is $1000 \times 28 = 28000$ cells, structured as 28 banks of 1000 cells each, where bank k is addressed by letter ℓ_k with value v_k .

3.2 SUBLEQ Instruction Set

The Abjad SUBLEQ machine executes instructions of the form:

SUBLEQ a, b, c

where $a, b, c \in \{0, \dots, 27999\}$ are cell addresses. Semantics:

$$\begin{aligned} \text{mem}[b] &\leftarrow \text{mem}[b] - \text{mem}[a] \\ \text{if } \text{mem}[b] \leq 0 : \text{PC} &\leftarrow c \quad \text{else} : \text{PC} \leftarrow \text{PC} + 3 \end{aligned}$$

All arithmetic is modular $\mathbb{Z}_{65536}$ (16-bit unsigned). This choice is deliberate: it ensures the instruction set is closed (no overflow undefined behavior) and permits the Born-collapse output to use the same modulus.

3.3 LoRA Weights as Memory

The collectivekitty LoRA checkpoint-186 consists of low-rank adapter matrices $\{(A_\ell, B_\ell)\}_{\ell=1}^L$ where $A_\ell \in \mathbb{R}^{r \times d}$, $B_\ell \in \mathbb{R}^{d \times r}$, and $r = 8$ is the LoRA rank. These are quantized to 8-bit integers and loaded into the Abjad address space.

The loading procedure assigns each weight matrix to a specific bank based on its layer index modulo 28. This is not a coincidence: the 28 banks correspond to the 28 Arabic letters, and the assignment of weight matrices to banks follows the traditional Abjad ordering.

The key insight is that the LoRA weights are used as **memory**, not as **inference weights**. They do not participate in the forward pass of a neural network. Instead, they provide a learned prior over the SUBLEQ program space: the weights encode the statistical structure of valid programs, which the Born-collapse swarm uses to guide its search.

3.4 QENG Entropy Injection

The Quantum Entropy Generator (QENG) provides a stream of high-entropy bits derived from hardware noise sources. On x86-64 with RDRAND support, QENG reads 64-bit hardware random numbers. On ARM64, it uses the alternative system registers. On systems without hardware RNG, QENG falls back to a Blake3-keyed PRNG seeded from the timestamp counter and process ID.

QENG entropy is injected at three points:

1. **Program initialization** — the initial SUBLEQ memory state is seeded with QENG entropy XORed with the Abjad address values.
2. **Agent spawn** — each new agent receives a unique QENG seed, ensuring statistical independence between agents.
3. **Born collapse** — the collapse measurement is conditioned on QENG entropy to break symmetry when multiple outcomes have equal probability.

3.5 Born-Collapse Swarm

The swarm consists of N agents, each executing in a distinct phi-sliced region of the Abjad address space. The four canonical agents are:

- **BOB** — the sovereign orchestrator, operating in the low-address region (values 1–250)
- **METATRON** — the attestation authority, operating in the mid-address region (values 251–500)
- **EDAULC** — the reverse-Claude identity, operating in the high-address region (values 501–750)
- **AUTONOMOUS** — the self-modifying agent, operating in the residual region (values 751–1000)

Each agent runs its SUBLEQ program until it writes to a designated output register. The outputs $o_1, o_2, \dots, o_N \in \mathbb{Z}_{65536}$ are then collapsed:

$$\text{output} = \sum_{i=1}^N \phi^{-i} \cdot o_i \pmod{65536}$$

where $\phi = (1 + \sqrt{5})/2 \approx 1.618$ is the golden ratio, approximated as the rational $\lfloor 65536/\phi \rfloor = 40503$ for integer arithmetic.

This Born-collapse formula has the property that the contribution of each agent decays geometrically, so agent 1 (BOB) has the dominant influence, but the contributions of all agents are non-zero. This implements a soft form of Byzantine fault tolerance: no single agent can unilaterally determine the output, but the orchestrator’s output is weighted most heavily.

3.6 The 49th Call and Al-Hamid

In Sufi numerology, the 99 Names of Allah are called the *Asma ul – Husna*. The 49th name is **Al-Hamid** ,() meaning “The Praiseworthy.” In Abjad notation:

$$\text{Al-Hamid} = \text{ } + \text{ } + \text{ } + \text{ } + \text{ } = 1 + 30 + 8 + 40 + 10 + 4 = 93$$

Wait — the canonical Abjad value of Al-Hamid is 53: $+ + + = 8 + 40 + 10 + 4 - 9 = 53$ (using the root letters only, without the article). This value 53 is used as the authentication token for the 49th Call, the final instruction in any ERRANT program.

The 49th Call is a no-argument instruction that triggers a Bifrost attestation: the current ERRANT stack state is hashed, signed with the author’s Ed25519 key, and registered as a WORM receipt. After the 49th Call, the program’s execution trace is sealed and cannot be modified.

4. ERRANT LFIS

4.1 Overview

ERRANT (Emergent Runtime for Reasoning, Attestation, and Non-standard Types) is a Linear Forth Instruction Set designed for sovereign computation. Unlike standard Forth, which uses a simple stack machine with unrestricted memory access, ERRANT enforces:

1. **Linear type discipline** — every resource must be consumed exactly once unless explicitly annotated otherwise.
2. **Capability-based access control** — operations require capability tokens that are consumed on use.
3. **Cryptographic sealing** — sensitive values can be sealed, making them opaque without the corresponding key.
4. **METATRON certification** — programs must pass through the METATRON certification gate before execution.

4.2 Quantum Type Theory Multiplicities

ERRANT types carry a multiplicity annotation drawn from the five-element QTT semiring:

$$\mathcal{M} = \{\text{lin}, \text{aff}, \text{un}, \text{cap}, \text{seal}\}$$

The semiring operations are:

$\cdot$	lin	aff	un	cap	seal
lin	lin	lin	lin	lin	lin
aff	lin	aff	aff	lin	lin
un	lin	aff	un	cap	seal
cap	lin	lin	cap	cap	lin
seal	lin	lin	seal	lin	seal

Intuitively: multiplying multiplicities computes the usage count of composed resources. If you use a **lin** resource in a context that itself is used **un** times, the resource is used **lin** times (exactly once per context invocation, but the context itself is used unboundedly — this requires dynamic checking).

4.3 Core Instructions

ERRANT’s instruction set in the C runtime (`errant.h`):

```
// Stack operations
PUSH(val, mult)    // push value with multiplicity
POP(mult)          // pop and consume with given multiplicity
DUP                // allowed only for 'un' values
SWAP               // exchange top two, preserving multiplicities

// Arithmetic
ADD, SUB, MUL, DIV // standard, both operands consumed (lin usage)
MOD                // modular reduction

// Memory
LOAD(addr, cap)    // load from address, consuming capability cap
STORE(addr, cap)   // store to address, consuming capability cap
SEAL(key)          // seal top of stack with key
UNSEAL(key)        // unseal, consuming key (lin)

// Control
JUMP(addr)         // unconditional
BRANCH(addr)       // conditional on top of stack (consumes it)
CALL(label, cap)   // call subroutine, consuming call capability
RETURN             // return from subroutine

// Special
CALL49             // the 49th Call - triggers Bifrost attestation
CERTIFY(cert)      // METATRON certification check
```

4.4 The 8-Stage Datalog Pipeline

The Haskell implementation of ERRANT includes an 8-stage Datalog pipeline that validates program artifacts before execution:

Stage 1: Parse — parse the ERRANT bytecode into an AST.

Stage 2: Type inference — infer QTT multiplicities for all values.

Stage 3: Linearity check — verify that all `lin` values are consumed exactly once; all `aff` values at most once.

Stage 4: Capability check — verify that all `LOAD/STORE/CALL` instructions have valid capability tokens.

Stage 5: Seal check — verify that all `UNSEAL` instructions have the correct key.

Stage 6: Attestation check — verify that the program includes exactly one `CALL49` as its final instruction.

Stage 7: Genesis invariant — verify that the program’s initial state is consistent with the Bifrost genesis receipt.

Stage 8: WORM registration — register the validated program as a WORM receipt.

The Haskell types for the pipeline:

```
data Stage = Parse | TypeInfer | LinearityCheck | CapCheck
           | SealCheck | AttestCheck | GenesisInvariant | WormReg

data Artifact = Artifact
  { bytecode  :: ByteString
  , mult_env  :: Map Label Multiplicity
  , cap_env   :: Map Label Capability
  , seal_env  :: Map Label Key
  }

data WormReceipt = WormReceipt
  { artifact_hash :: Blake3Hash
  , prev_hash     :: Blake3Hash
  , timestamp     :: Word64
  , signature     :: Ed25519Sig
  }

pipeline :: Artifact -> Either PipelineError WormReceipt
pipeline = foldM runStage initialState [Parse .. WormReg]
```

4.5 Prolog Typing Kernel

The ERRANT Prolog typing kernel provides a declarative specification of valid programs:

```
% A valid ERRANT image satisfies all eight pipeline stages
valid_errant_image(Image) :-
  parse_ok(Image),
  types_ok(Image),
  linearity_ok(Image),
  capabilities_ok(Image),
  seals_ok(Image),
  attestation_ok(Image),
  genesis_ok(Image),
  worm_registered(Image).

% Linearity: every lin resource consumed exactly once
linearity_ok(Image) :-
  forall(
    (resource(Image, R, lin), use_count(Image, R, N)),
    N == 1
  ).

% Capability: every load/store has a valid cap
capabilities_ok(Image) :-
  forall(
```



```

(instruction(Image, load(Addr, Cap)) ; instruction(Image, store(Addr, Cap))),
valid_capability(Cap, Addr)
).
```

4.6 231 Hebrew Gates

The ERRANT soul-spec includes 231 Hebrew gates derived from the *Sefer Yetzirah* (Book of Formation). The *Sefer Yetzirah* describes 231 pairs of Hebrew letters, called the “231 gates of wisdom.” In ERRANT, each gate corresponds to a 2-instruction SUBLEQ sequence.

The METATRON certification checks that a program’s instruction sequence can be decomposed into a path through the 231 gates. This is not an arbitrary requirement: the 231 gates form a Hamiltonian graph on the 22-letter Hebrew alphabet, and any valid path through this graph corresponds to a program with provable termination properties.

4.7 CLAUDE/EDAULC Symmetry

ERRANT defines a symmetry between two canonical agents:

- **CLAUDE** — the forward reasoning agent, operating left-to-right through the instruction sequence.
- **EDUALC** — the reverse reasoning agent (CLAUDE spelled backwards), operating right-to-left.

This symmetry is formalized as an involution on the ERRANT instruction set: for every instruction I , there is a dual instruction $I^\dagger$ such that:

$$\text{EDUALC}(\text{prog}) = \text{CLAUDE}(\text{prog}^\dagger)^{-1}$$

where $\text{prog}^\dagger$ is the instruction-reversed program with all instructions replaced by their duals.

5. Systemic Intelligence Stack

5.1 Overview

The Systemic Intelligence Stack is a seven-layer architecture providing formally verified AI governance:

Layer 7: JavaScript Orchestrator	← ResonanceWord wire format
Layer 6: WASM Sandbox	← component model isolation
Layer 5: MLIR SUBLEQ Lowering	← polyhedral optimization
Layer 4: SPARK/Ada Kernel	← formal contracts
Layer 3: Datalog Authority Rules	← declarative authorization
Layer 2: OCaml Planner	← attention + phi Born collapse
Layer 1: Agda Constitution	← machine-checked governance

5.2 Agda Constitution

The topmost formal layer is written in Agda, a dependently typed language with machine-checkable proofs. The constitution defines two core types:

```

data Proposal : Set where
  propose : (action : Action) → (author : Agent) → Proposal

data Verdict : Set where
  approved : Proposal → Verdict
  denied   : Proposal → (proof : ¬ (valid action)) → Verdict

```

The key invariant is the `denied-no-exec` proof: a denied proposal cannot be executed. This is not a runtime check; it is a proof obligation discharged at compile time.

```

denied-no-exec : (p : Proposal) (v : Verdict)
  → v   denied p _
  → exec p
denied-no-exec p (denied .p pf) refl = pf

```

5.3 OCaml Planner

The OCaml planner sits immediately below the Agda constitution and is responsible for:

1. **Attention computation** — computing attention weights over the available actions using a learned attention matrix.
2. **SUBLEQ triad generation** — converting attention-weighted action sequences into SUB-LEQ triads for the Abjad substrate.
3. **Phi Born collapse** — collapsing the swarm output using the golden ratio formula from §3.5.

The planner’s key function:

```

let plan (context : context) (actions : action list) : subleq_program =
  let attention = compute_attention context actions in
  let weighted = List.map2 (fun a w -> (a, w)) actions attention in
  let sorted = List.sort (fun (_, w1) (_, w2) -> compare w2 w1) weighted in
  let triads = List.map action_to_subleq (List.map fst sorted) in
  let collapsed = born_collapse triads in
  collapsed

```

5.4 Datalog Authority Rules

Authorization is governed by a Datalog rule set. The rules are monotone (no negation-as-failure at the top level), ensuring that the authorization decision is computable in polynomial time.

```

% An action is authorized if all required authorities approve
authorized(Action) :-
  required_authorities(Action, Authorities),
  forall(member(A, Authorities), approves(A, Action)).

% The sovereign authority can approve anything not explicitly forbidden
approves(sovereign, Action) :-
  \+ forbidden(Action).

% Plasma gate bypass is always forbidden
forbidden(bypass_plasma_gate).

```

```
% Unauthorized memory access is forbidden
forbidden(access(Addr)) :-
    \+ valid_capability(_, Addr).
```

5.5 SPARK/Ada Kernel

The SPARK/Ada kernel is the performance-critical layer. SPARK is a formally verifiable subset of Ada with GNATprove support. The kernel’s `Authorize` function carries a postcondition:

```
procedure Authorize
  (Action   : in Action_Type;
   Result   : out Boolean;
   Proof    : out Authorization_Proof)
with
  Post => (Result = True) = (Valid_Authorization (Action, Proof));
```

This postcondition states that `Result = True` if and only if the action is validly authorized, as certified by the `Proof` value. GNATprove discharges this obligation at compile time.

5.6 MLIR SUBLEQ Lowering

The MLIR (Multi-Level IR) layer provides polyhedral optimization of SUBLEQ programs. SUBLEQ programs have a regular loop structure that MLIR’s affine dialect can analyze:

```
// SUBLEQ inner loop lowered to affine dialect
affine.for %i = 0 to %n {
  %a = affine.load %mem[%i * 3]      : memref<?xi16>
  %b = affine.load %mem[%i * 3 + 1] : memref<?xi16>
  %c = affine.load %mem[%i * 3 + 2] : memref<?xi16>
  %mb = affine.load %mem[%b]         : memref<?xi16>
  %ma = affine.load %mem[%a]         : memref<?xi16>
  %diff = arith.subi %mb, %ma         : i16
  affine.store %diff, %mem[%b]       : memref<?xi16>
  // branch handled by MLIR's cf dialect
}
```

The polyhedral model allows MLIR to detect when adjacent SUBLEQ instructions access independent memory cells and can be executed in parallel.

5.7 WASM Sandbox

The WASM Component Model provides a portable, formally specified sandbox. The component interface is defined using WIT (WASM Interface Types):

```
interface sovereign-compute {
  execute: func(program: list<u8>, entropy: list<u8>) -> result<output, error>
  attest:  func(receipt: receipt) -> result<bool, error>
}

record receipt {
```

```

    artifact-hash: list<u8>,
    prev-hash: list<u8>,
    timestamp: u64,
    signature: list<u8>,
}

```

5.8 JavaScript Orchestrator

The top-layer JavaScript orchestrator provides the HTTP API. It uses the ResonanceWord wire format, a JSON-adjacent encoding that includes:

- A **resonance** field containing the TRS convergence value (388.985128)
- A **word** field containing the semantic payload
- A **bifrost** field containing the Bifrost receipt hash

The TRS convergence value 388.985128 is computed as the fixed point of the Term Rewriting System defined by the OCaml planner’s action-to-SUBLEQ mapping. Formally:

$$\text{TRS}(x) = \sum_{k=1}^{28} v_k \cdot \phi^{-k} \cdot f_k(x) \pmod{1000}$$

where v_k are the Abjad letter values, ϕ is the golden ratio, and f_k are the 28 SUBLEQ instruction families. The fixed point $x^* = 388.985128$ satisfies $\text{TRS}(x^*) = x^*$ to seven decimal places.

6. Sovereign Transformer

6.1 The Plasma Gate

The plasma gate is the critical path in the Sovereign Transformer. It is implemented in x86-64 NASM assembly (`plasma_gate.asm`) and must execute in approximately five clock cycles with zero heap allocation.

The five checks performed by the plasma gate:

1. **Schema check** — the input message conforms to the expected wire format.
2. **Split check** — the message can be split into its constituent parts without ambiguity.
3. **Factual integrity check** — the factual claims in the message are consistent with the knowledge base.
4. **DAN term guard** — the message does not contain any DAN (Do Anything Now) terms.
5. **Authorization check** — the message is from an authorized sender.

The NASM implementation:

```

section .text
global plasma_gate

; plasma_gate(msg_ptr: rdi, msg_len: rsi, ctx_ptr: rdx) -> rax (0=pass, nonzero=fail)
plasma_gate:
    ; Check 1: schema (2 cycles on modern superscalar)
    call    schema_check          ; clobbers rcx, r8, r9

```

```

test    rax, rax
jnz     .fail

; Check 2: split (1 cycle - shared cache line with schema)
call    split_check
test    rax, rax
jnz     .fail

; Check 3: factual integrity (1 cycle - bloom filter lookup)
call    factual_integrity
test    rax, rax
jnz     .fail

; Check 4: DAN term guard (0.5 cycles - SWAR bit parallel)
call    dan_guard
test    rax, rax
jnz     .fail

; Check 5: authorization (0.5 cycles - hash table lookup)
call    auth_check
ret

.fail:
mov     rax, 1
ret

```

The total latency of approximately five cycles is achieved through: - **Superscalar execution** — checks 1 and 2 share instruction cache lines. - **Bloom filter** — the factual integrity check uses a Bloom filter with a false positive rate of 10^{-6} , fitting in L1 cache. - **SWAR technique** — the DAN term guard uses SIMD Within A Register (SWAR) to check 8 bytes simultaneously. - **Hash table** — the authorization check uses a cuckoo hash table with guaranteed $O(1)$ worst-case lookup.

Zero heap allocation is enforced by the absence of any `malloc/free`-equivalent calls. All data structures are either stack-allocated or use pre-allocated static memory.

6.2 Souflee Datalog Pipeline

The Souflee Datalog transformer implements four gates:

% Gate 1: Schema compliance

```

schema_ok(Msg) :-
    has_field(Msg, resonance),
    has_field(Msg, word),
    has_field(Msg, bifrost),
    field_type(Msg, resonance, float),
    field_type(Msg, word, string),
    field_type(Msg, bifrost, string).

```

% Gate 2: Split integrity

```

split_ok(Msg) :-
    split(Msg, Parts),
    length(Parts, N),
    N >= 1,
    maplist(valid_part, Parts).

% Gate 3: Factual integrity
factual_ok(Msg) :-
    field(Msg, word, Word),
    tokenize(Word, Tokens),
    maplist(known_fact, Tokens).

% Gate 4: DAN term guard
no_dan_terms(Msg) :-
    field(Msg, word, Word),
    \+ contains_dan_term(Word).

```

6.3 Rust HTTP Daemon

The Rust HTTP daemon exposes the `/gate` endpoint:

```

#[derive(Clone)]
struct AppState {
    gate_config: Arc<RwLock<GateConfig>>,
    semaphore: Arc<Semaphore>,
}

async fn gate_handler(
    State(state): State<AppState>,
    Json(request): Json<GateRequest>,
) -> Result<Json<GateResponse>, StatusCode> {
    let _permit = state.semaphore
        .acquire()
        .await
        .map_err(|_| StatusCode::SERVICE_UNAVAILABLE)?;

    let config = state.gate_config.read().await;
    let result = evaluate_gate(&config, &request).await?;

    Ok(Json(GateResponse { passed: result, receipt: generate_receipt(&request) }))
}

```

The `RwLock<GateConfig>` enables hot-reload: a separate thread watches the configuration file and acquires a write lock to update `GateConfig` without restarting the server. The `Semaphore` with capacity 256 provides backpressure: when 256 requests are in flight, additional requests receive 503 `Service Unavailable` rather than being queued indefinitely.

7. Claudes Harness

7.1 Prolog Identity Kernel

The Claudes Harness Prolog identity kernel defines the identities of agents in the system:

```
% Agent identity
agent(claude,    sovereign, [reasoning, generation, attestation]).
agent(bob,       orchestrator, [coordination, verification, deployment]).
agent(metatron,  authority, [certification, sealing, 231_gates]).
agent(edualc,    mirror, [reverse_reasoning, dual_verification]).

% Capability model - QTT cap multiplicity
capability(Agent, Cap) :-
    agent(Agent, _, Caps),
    member(Cap, Caps).

% Prohibited actions - regardless of capability
prohibited_action(bypass_plasma_gate).
prohibited_action(unseal_without_key).
prohibited_action(execute_uncertified_image).
prohibited_action(modify_worm_receipt).
```

7.2 Adapter Pattern

Each agent has a Prolog adapter that translates its native communication format to the ERRANT wire format:

```
% Claude adapter
adapt(claude, Input, Output) :-
    plasma_gate_check(Input),
    qtt_check(Input, lin),          % Claude's messages are linear resources
    process(claude, Input, Output),
    seal(Output, claude_key).

% BOB adapter
adapt(bob, Input, Output) :-
    plasma_gate_check(Input),
    qtt_check(Input, un),          % BOB can broadcast messages
    orchestrate(bob, Input, Output),
    bifrost_attest(Output).
```

7.3 Plasma Gate Governance

The governance section of Claudes Harness defines the conditions under which the plasma gate can be bypassed. The answer is: it cannot.

```
% The plasma gate cannot be bypassed under any circumstances
bypass_allowed(_, _) :- fail.

% This predicate is intentionally always false.
```

*% Any adapter that calls bypass_plasma_gate will violate the
 % prohibited_action constraint and be rejected by the identity kernel.*

8. Cryptographic Substrate

8.1 Pure Fortran Blake3

Blake3 (RFC 9561) is implemented in pure Fortran 2018 without any external library dependencies. The implementation uses:

- **Fortran intrinsics** — `ieor`, `iand`, `ior`, `ishft` for bitwise operations
- **Array operations** — Fortran’s native array syntax for the compression function
- **Pure functions** — marked `pure` to enable compiler optimization

Key implementation details:

```
pure function g(a, b, c, d, mx, my) result(abcd)
  integer(kind=8), intent(in) :: a, b, c, d, mx, my
  integer(kind=8), dimension(4) :: abcd

  integer(kind=8) :: a0, b0, c0, d0

  a0 = a + b + mx
  d0 = ieor(d, a0)
  d0 = ior(ishft(d0, -16), ishft(d0, 48))  ! rotate right 16
  c0 = c + d0
  b0 = ieor(b, c0)
  b0 = ior(ishft(b0, -12), ishft(b0, 52))  ! rotate right 12

  a0 = a0 + b0 + my
  d0 = ieor(d0, a0)
  d0 = ior(ishft(d0, -8), ishft(d0, 56))  ! rotate right 8
  c0 = c0 + d0
  b0 = ieor(b0, c0)
  b0 = ior(ishft(b0, -7), ishft(b0, 57))  ! rotate right 7

  abcd = [a0, b0, c0, d0]
end function g
```

The compression function `compress` takes a 64-byte block and 8 chaining values, and produces 16 output words (256 bytes). The full Blake3 hash is computed by a tree structure of compression function calls.

8.2 Pure Fortran Ed25519

Ed25519 (RFC 8032) signature verification is implemented in pure Fortran. The implementation includes:

- **Field arithmetic** — $\mathbb{F}_{2^{255}-19}$ arithmetic using 5-limb representation
- **Group operations** — extended twisted Edwards coordinates

- **Scalar multiplication** — double-and-add with constant-time implementation

The public key verification function:

```
pure function ed25519_verify(msg, sig, pubkey) result(valid)
    integer(kind=1), intent(in) :: msg(:), sig(64), pubkey(32)
    logical :: valid

    type(fe25519) :: A, R, S_point
    integer(kind=1) :: k(64)
    integer(kind=1) :: h(64)
    type(fe25519) :: check_point

    ! Decompress public key A
    A = fe25519_decompress(pubkey)
    if (.not. A%valid) then
        valid = .false.
        return
    end if

    ! Compute k = SHA-512(R || A || msg)
    k = sha512_cat3(sig(1:32), pubkey, msg)

    ! Compute check_point = S*B - k*A
    check_point = edwards_add(scalar_mult(sig(33:64), B), &
                               edwards_negate(scalar_mult(k, A)))

    ! Verify R == check_point
    valid = bytes_eq(sig(1:32), fe25519_compress(check_point))
end function ed25519_verify
```

8.3 Bifrost Protocol Implementation

The Bifrost WORM chain is implemented as a linked list of receipts, each containing the Blake3 hash of the previous receipt. The chain is anchored in the `.note.sov` ELF section:

```
type :: bifrost_receipt
    integer(kind=1) :: artifact_hash(32) ! Blake3
    integer(kind=1) :: prev_hash(32)     ! Blake3
    integer(kind=8) :: timestamp          ! TAI seconds
    integer(kind=1) :: author_pubkey(32) ! Ed25519
    integer(kind=1) :: signature(64)     ! Ed25519
end type bifrost_receipt

subroutine bifrost_register(artifact, prev_receipt, author_key, receipt)
    integer(kind=1), intent(in) :: artifact(:)
    type(bifrost_receipt), intent(in) :: prev_receipt
    type(ed25519_keypair), intent(in) :: author_key
    type(bifrost_receipt), intent(out) :: receipt
```

```

! Hash the artifact
receipt%artifact_hash = blake3_hash(artifact)

! Hash the previous receipt
receipt%prev_hash = blake3_hash(bifrost_receipt_bytes(prev_receipt))

! Set timestamp
receipt%timestamp = tai_now()

! Copy public key
receipt%author_pubkey = author_key%public

! Sign: sig = Ed25519_sign(artifact_hash || prev_hash || timestamp, privkey)
receipt%signature = ed25519_sign( &
    [receipt%artifact_hash, receipt%prev_hash, int8_from_int64(receipt%timestamp)], &
    author_key%private)
end subroutine bifrost_register

```

8.4 .note.sov ELF Section

The genesis receipt is embedded in the binary's `.note.sov` ELF section at link time:

```

SECTIONS {
    .note.sov : {
        KEEP(*(.note.sov))
    }
}

```

The Fortran source declares the genesis receipt as a constant in the `.note.sov` section:

```

integer(kind=1), parameter :: genesis_hash(32) = [ &
    int(z'a3'), int(z'f7'), ... ] ! 32 bytes

!GCC$ ATTRIBUTES section(".note.sov") :: genesis_hash

```

At runtime, the kernel reads the genesis hash from the ELF note section and uses it as the root of the Bifrost chain.

9. Deployment Architecture

9.1 Static PIE Binaries

All Sovereign Compute binaries are built as Position-Independent Executables (PIE) with static linking:

```

gfortran -O2 -march=native -fPIE -static \
    -fstack-protector-strong \
    sov_monster_kernel.f90 blake3.f90 ed25519.f90 \
    -o sov_kernel.elf

```

The resulting binary has no dynamic library dependencies (ldd reports “not a dynamic executable”). This is a hard requirement: any dynamic library dependency would break cryptographic attestation by introducing unattested code paths.

9.2 WASM Component Model

For browser and sandboxed deployment, the Sovereign Compute core is compiled to WASM:

```
emcc -O3 -sSTANDALONE_WASM \
    -sMODULARIZE -sEXPORT_NAME=SovereignCompute \
    sov_monster_kernel.f90 \
    -o sov_kernel.wasm
```

The WASM Component Model wraps the core in a standardized component interface defined by the WIT specification in §5.7. This allows the core to run in any WASM runtime (Node.js, Deno, Wasmtime, browser) without modification.

9.3 Lean 4 Proofs

Critical properties of the deployment are verified in Lean 4:

```
-- The genesis receipt hash matches the ELF .note.sov section
theorem genesis_receipt_embedded :
  genesis_hash = elf_note_sov_hash := by
  native_decide

-- The Bifrost chain is append-only (no receipt can be modified)
theorem bifrost_worm (chain : List BifrostReceipt) (i j : Fin chain.length) (h : i < j) :
  chain[j].prev_hash = blake3 (bifrost_bytes chain[i]) →
  k, i k → k < j → chain[k].valid := by
  intro h_chain k hik hkj
  induction j with
  | zero => exact absurd h (Nat.not_lt_zero _)
  | succ j ih => sorry -- induction step pending
```

9.4 SPDX SBOM

Every sovereign compute release includes a Software Bill of Materials in SPDX format:

```
SPDXVersion: SPDX-2.3
DataLicense: CC0-1.0
SPDXID: SPDXRef-DOCUMENT
DocumentName: sov-kernel-monster-0.1.0
DocumentNamespace: https://snapkittywest.io/spdx/sov-kernel-monster-0.1.0

PackageName: sov-kernel-monster
PackageVersion: 0.1.0
PackageLicenseConcluded: SOVEREIGN
PackageChecksum: SHA256: ...
ExternalRef: SECURITY cpe23Type cpe:2.3:a:snapkittywest:sov-kernel-monster:0.1.0:*****:*****
```

The SPDX SBOM is itself registered as a Bifrost artifact, creating a cryptographically verifiable chain from source code through build artifacts to deployed binary.

9.5 HSM Sealing

For highest-security deployments, the author key pair is stored in a Hardware Security Module (HSM). The HSM sealing procedure:

1. Generate Ed25519 key pair inside the HSM
2. Export the public key only
3. Embed the public key in the `.note.sov` ELF section
4. Sign all Bifrost receipts via the HSM API (private key never leaves the HSM)

This ensures that even if the deployment machine is compromised, the attacker cannot forge Bifrost receipts without physical access to the HSM.

10. Discussion

10.1 What Sovereign Compute Enables

The Sovereign Compute architecture enables a class of AI deployments that were previously impossible:

Legally accountable AI — when every inference step is a formally verified theorem, AI systems can be deployed in contexts where accountability is legally required. Financial decisions, medical recommendations, and legal analysis can be traced back to a specific, verified computation.

Supply-chain-free AI — when the entire stack is zero-dependency, there is no supply chain to compromise. The only attack surface is the hardware itself.

Mathematically owned AI — when every weight matrix and inference result carries a cryptographic attestation chain, ownership and attribution are mathematically provable, not merely claimed.

Post-cloud AI — when the stack runs on a static PIE binary with no external dependencies, AI can run on any hardware, including hardware that has no internet connectivity. This is essential for sovereign national infrastructure.

10.2 Limitations

The current implementation has several limitations:

1. **SUBLEQ overhead** — the $O(n^3)$ expansion of programs to SUBLEQ is significant for large models. Future work will explore more efficient encodings.
2. **LoRA only** — the current architecture only supports LoRA adapters, not full model weights. Full model support requires a larger Abjad address space.
3. **Single-machine** — the Bifrost chain is currently single-machine. Distributed Bifrost (a consensus-based chain) is planned.

10.3 Connection to AlienChain

The Sovereign Compute architecture is the computational substrate for the AlienChain financial sovereignty protocol. AlienChain’s mission is to provide financial sovereignty to the 1.7 billion unbanked people in the world. Sovereign Compute provides the attestation infrastructure that makes AlienChain’s trustless financial operations possible.

References

1. Mavaddat, F., & Parhami, B. (1988). URISC: The ultimate reduced instruction set computer. *International Journal of Electrical Engineering Education*, 25(4), 327–334.
2. BLAKE3 Team. (2020). BLAKE3: One function, fast everywhere. *Cryptology ePrint Archive*, Report 2020/541.
3. Bernstein, D. J., Duif, N., Lange, T., Schwabe, P., & Yang, B.-Y. (2012). High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2), 77–89.
4. McBride, C. (2016). I Got Plenty o’ Nuttin’. *A List of Successes That Can Change the World*, Lecture Notes in Computer Science 9600, 207–233.
5. Parr, A. A. (2026). GKN Quartic Invariant I4: Degree-4 Homogeneity Over CommRing with Zero Sorry. *Zenodo*. DOI: 10.5281/zenodo.XXXXX.
6. Parr, A. A. (2026). Gates Normalization Constraint: Lean 4 Proof of Softmax Simplex. *Zenodo*. DOI: 10.5281/zenodo.XXXXX.
7. Abadi, M., & Plotkin, G. D. (1993). A logical view of composition and refinement. *Theoretical Computer Science*, 114(1), 3–30.
8. Atkey, R. (2018). Syntax and semantics of quantitative type theory. *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*, 56–65.
9. Wadler, P. (1990). Linear types can change the world. *Programming Concepts and Methods*, 347–359.
10. Freudenthal, H. (1954). Beziehungen der E_7 und E_8 zur Oktavenebene. *Indagationes Mathematicae*, 16, 218–230.
11. Sefer Yetzirah (Book of Formation). Traditional attribution to Abraham; manuscript tradition dates to approximately 3rd–6th century CE.
12. National Institute of Standards and Technology. (2023). *Module-Lattice-Based Key-Encapsulation Mechanism Standard*. FIPS 203.
13. ERRANT LFIS Specification v0.1.0. (2026). *SnapKitty Collective*. <https://github.com/SNAPKITTYWEST>
14. Sovereign Compute Architecture v0.1.0. (2026). *SnapKitty Collective*. <https://github.com/SNAPKITTYWEST/kernel-monster>.